

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 2 of 22 — Foundations

**Python Zero Se Shuru: Coding Ki ABC, Apni Hi Research Ke Asli
Code Se**

Sakshi ke liye, Sakshi ki apni research par — samjhaya gaya ekdum scratch se,
taaki BORDER OPTICS ke code-heavy PDFs (PDF 6 se aage) mein syntax kabhi roke
nahi.

Is PDF mein: Python program kya hota hai, variable/comment/indentation ki ABC,
lists/dictionaries/tuples/sets, if-else aur loops, functions, libraries import karna, aur
sabse zaroori — pandas dataframe (jo BORDER OPTICS ke har script mein use hota hai).
Har concept BORDER OPTICS ke asli code se, generic textbook examples se nahi —
kyunki end goal Python seekhna nahi, BORDER OPTICS samajhna hai. Aakhir mein ek
poori, chhoti, asli research file (merge_geocoded.py) line-by-line padhenge, taaki sab
kuch jud jaaye.

Shuru Karte Hain — Python Hai Kya, Aur Ye BORDER OPTICS Mein Kyun Hai

PDF 1 mein humne dekha tha ki BORDER OPTICS satellite se numbers nikaalta hai — 258 gaon, do proxies (NDBI, lights), do compositing windows, teen time-points. Ye saari calculation kisi ne haath se, calculator leke nahi ki — ek program ne ki, jo automatically, bina thake, exactly wahi steps hazaron baar repeat kar sakta hai jo humne use karne ko bola. Wo program jis language mein likha gaya hai, uska naam hai Python.

Python ek programming language hai — matlab, instructions likhne ka ek tareeka jo computer samajh sakta hai. Jaise hum kisi insaan ko Hindi ya English mein instruction dete hain ('pehle ye karo, fir wo karo'), waise hi Python mein hum computer ko exact, step-by-step instructions dete hain, lekin ek bahut strict grammar ke saath — ek chhota sa spelling ya spacing ka farak bhi computer ko confuse kar sakta hai. Yahi strictness hi is PDF ka poora point hai — jab hum PDF 6 se BORDER OPTICS ke asli code files padhna shuru karenge, tujhe ye grammar already pata honi chahiye, taaki tu sirf 'ye line research mein kya kar rahi hai' par focus kar sake, na ki 'ye bracket yahan kyun hai' par atak jaaye.

BORDER OPTICS mein total 45 Python files hain — chhote (50-100 lines) se lekar bade (select_control_villages.py akela 18,000+ characters ka) tak. Sab ek hi language (Python) mein likhe gaye hain kyunki Python data science aur research ke liye duniya bhar mein sabse zyada use hone waali language hai — iski wajah simple hai: Python ke paas libraries (pehle se likhe gaye, ready-to-use code ke packages) hain jo satellite data (Google Earth Engine), tables/spreadsheets (pandas), aur statistics (scipy) jaisi cheezon ko handle karna bahut aasan bana dete hain. In libraries ko import karna, aur inko use karna, hum is PDF mein dekhenge.

EXAMPLE — Recipe Ki Analogy

Python code ek recipe jaisa hai. Recipe mein steps hote hain, top se neeche, ek order mein follow karne ke liye — 'pehle pyaaz kaato, fir tel garam karo, fir pyaaz daalo'. Agar order badal do (tel garam karne se pehle hi pyaaz daal do bina tel ke), result galat aayega. Python program bhi bilkul aisa hi hai — computer ek-ek line, top se neeche, exact order mein follow karta hai. Farak sirf itna hai ki recipe mein thodi flexibility chalti hai ('ya thoda zyada namak daal do'), Python mein bilkul nahi — ek chhoti si galti (spelling, spacing, bracket) poora program crash kar sakti hai.

1. Ek Python File Kaise Chalti Hai

BORDER OPTICS ki har .py file ek text file hai jisme Python instructions likhi hain, top se neeche. Jab koi ise 'run' karta hai (jaise terminal mein python extract_satellite_data.py type karke), Python interpreter (ek software jo Python samajhta hai) file ko upar se neeche, line by line padhta aur execute karta hai — bilkul jaise koi kitaab top se neeche padhi jaati hai, ek baar mein ek line.

Ismein kuch exceptions hain jo hum aage dekhenge — jaise functions (jo tab tak nahi chalte jab tak unhe 'bulaya' na jaaye) aur loops (jo ek hi block ko baar-baar repeat karte hain) — lekin basic rule yahi hai: top-to-bottom, ek line ek waqt mein. Ye samajhna zaroori hai kyunki jab hum PDF 6 se asli files padhenge, hum bhi unhe isi order mein, top se neeche, samjhenge.

2. Comments — Insaan Ke Liye Notes, Computer Ke Liye Invisible

Python code mein hum kabhi-kabhi aisi lines likhte hain jo Python bilkul ignore kar deta hai — ye sirf insaanon ke padhne ke liye hoti hain, jaise ek note. Inhe comments kehte hain, aur ye do tareeke se likhe jaate hain.

```
# Ye ek single-line comment hai — # ke baad jo kuch bhi likha ho, Python usko ignore karta hai
BUFFER_RADIUS_M = 500 # yahan bhi comment lag sakta hai, line ke aakhir mein
""" Ye ek multi-line comment/docstring hai, triple-quotes (""") mein. Zyadatar file ke bilkul shuru mein use hota hai, poori file ka purpose batane ke liye. """
```

BORDER OPTICS ki har file ke bilkul upar aisa hi ek triple-quote docstring milega — jaise merge_geocoded.py ke upar likha hai: `"""BORDER OPTICS — Merge all geocoded villages into one master table for upload to Google Earth Engine."""`. Ye line Python ke liye kuch nahi karti (koi calculation nahi, koi variable nahi), lekin insaan ke liye — tere jaisi kisi ke liye jo file khol ke samajhna chahti hai ki ye kis liye hai — ye pehla, sabse zaroori clue hai.

EXAMPLE — Kyun Zaroori Hai Comments Ka?

Socho tu 6 mahine baad apni khud ki likhi file dobara kholti hai. Bina comments ke, tujhe har line ko dobara samajhna padega. Comments ke saath, tu turant yaad kar sakti hai 'ohh haan, ye line isliye hai'. BORDER OPTICS ke code mein comments bahut strategically use hue hain — jaise compute_border_distance.py mein ek comment hai: `# field names vary by Natural Earth version, check columns if this filter comes up empty` — ye ek future-warning hai, jo batata hai ki agar kabhi ye specific line error de, toh sabse pehle kahan dekhna hai.

3. Variables — Data Ko Naam Dena

Ek variable Python mein ek 'box' jaisa hai jisme tu koi value store kar sakti hai, aur us box ko ek naam de sakti hai taaki aage use kar sake. Python mein variable banana bahut simple hai — bas naam likho, ek = (equals sign) lagao, aur value likho.

```
BUFFER_RADIUS_M = 500 VILLAGES_PATH =  
"data/processed/border_optics_master_villages.csv" S2_COLLECTION =  
"COPERNICUS/S2_SR_HARMONIZED"
```

Ye teeno lines asli `extract_satellite_data.py` file se hain (PDF 1 mein hum ye file already thodi dekh chuke hain). Pehli line ek variable `BUFFER_RADIUS_M` banati hai jisme number 500 store hai — yahi wo 500m buffer radius hai jo humne PDF 1 mein detail se samjha tha. Doosri aur teesri line mein string values (text, quotes ke andar) store hain — ek file ka path, aur Google Earth Engine ke satellite collection ka naam.

Python mein data ke kuch basic types hote hain jo tujhe har jagah dikhenge: int (poora number, jaise 500), float (decimal number, jaise 0.157), str (string, text, hamesha quotes `"..."` ya `'...'` mein), aur bool (boolean, sirf do values: True ya False). Ek variable jo abhi khaali/kuch nahi represent karta, usko None se assign kiya jaata hai (`extract_satellite_data.py` mein hi, agar kisi village ke liye koi valid image na mile, toh uski value None set hoti hai — null, zero nahi, jaisa PDF 1 mein bhi zikr hua tha).

EXAMPLE — Labeled Dabbon Ki Analogy

Socho ek almari mein bahut saare dabbe hain, aur har dabbe par ek label chipka hai. `BUFFER_RADIUS_M` naam ka dabba hai jisme '500' likha kaagaz rakha hai. Jab bhi code mein aage `BUFFER_RADIUS_M` naam likha milega, Python us dabbe ko khol ke '500' nikaal lega. Agar tu us dabbe ki value badalna chahe (jaise buffer 250m karna ho), tujhe sirf ek jagah — jahan variable define hua hai — number badalna padega, poori file mein har jagah dhundh ke badalna nahi padega. Yahi variable ka sabse bada fayda hai.

3.1 Naming Convention — Naam Kaise Rakhte Hain

Note kar: `BUFFER_RADIUS_M` sab CAPITAL letters mein hai — ye Python ki ek widely-followed convention hai jo batati hai ki ye ek constant hai, matlab ek value jo poori file mein kabhi nahi badalegi. Normal variables (jo badal sakte hain) chhote letters mein likhe jaate hain, jaise `villages`, `count`, `ndbi_before`. Underscores (`_`) words ko alag karne ke liye use hote hain, kyunki Python variable names mein spaces allow nahi karta.

4. Indentation — Python Ka Sabse Zaroori, Sabse Alag Rule

Ab ek aisa concept jo Python ko bahut si doosri languages se alag banata hai, aur jo BORDER OPTICS ke har single file mein baar-baar dikhega: indentation — matlab line ke shuru mein kitna khaali space (spaces ya tab) hai.

Zyadatar languages (jaise C, Java) mein code ke 'blocks' (jaise 'ye sab lines ek if-condition ke andar hain') curly braces { } se mark kiye jaate hain, aur indentation sirf padhne mein aasani ke liye hoti hai, Python ki tarah zaroori nahi. Python mein indentation hi block ko define karti hai — agar do lines ka indentation same hai, wo ek hi block ka hissa hain. Agar indentation badal jaaye, block khatam ho jaata hai.

```
if count == 0: return None, 0
composite = collection.median()
ndbi_image = composite.normalizedDifference(["B11", "B8"]).rename("NDBI")
```

Is asli snippet mein (extract_satellite_data.py se), return None, 0 line if count == 0: ke andar hai — isliye wo 4 spaces se indented (andar dhakeli hui) hai. Agla do lines (composite = ... aur ndbi_image = ...) bilkul indentation ke bina hain — matlab wo if ke andar nahi, balki us se bahar, normal flow mein hain, hamesha chalengi chahe count == 0 true ho ya false.

EXAMPLE — Ek Ghar Ke Kamron Ki Analogy

Socho ek ghar hai. Bahar ka aangan 'no indentation' hai — sab yahan tak pahunchte hain. Ek kamra (jaise 'agar mehmaan hai toh') ek level indent hai — sirf tab andar jaate hain jab condition sahi ho. Us kamre ke andar ek almari (indent ka aur ek level) — aur bhi zyada specific. Jitna zyada indent, utna 'zyada andar', utni specific condition ke andar wo line chalti hai. Jaise hi indentation kam hoti hai, tu wapas bahar aa jaati hai — us specific condition se bahar.

Python mein standard practice hai 4 spaces per indentation level (tab nahi, spaces — mixing tabs aur spaces se error aa sakta hai). Ye itna strict rule hai ki agar galti se ek space bhi idhar-udhar ho jaaye, Python IndentationError dega aur poora program rukk jaaega. Ye sunne mein strict lagta hai, lekin isi strictness ki wajah se Python code padhne mein bahut clean aur organized dikhta hai — indentation khud hi batati hai kaun si line kiske 'andar' hai, bina kisi curly-brace-counting ke.

5. Lists — Cheezon Ki Ek Ordered List

Ek list Python mein kai values ko ek saath, ek order mein, ek hi variable mein store karne ka tareeka hai. List [] (square brackets) mein likhi jaati hai, values ke beech comma , se.

```
outcome_cols = [ "ndbi_before", "ndbi_after", "ndbi_before_image_count",
"ndbi_after_image_count", "lights_before", "lights_after", "lights_before_image_count",
"lights_after_image_count", ]
```

Ye asli line `extract_satellite_data.py` se hai — ek list jisme 8 column names (sab strings) hain, jo baad mein use hoti hain ye check karne ke liye ki har gaon ke ye 8 columns already filled hain ya nahi. List ke andar ke items order mein rehte hain (pehla item hamesha pehla rahega), aur unko unki position (index) se access kiya ja sakta hai — Python mein counting 0 se shuru hoti hai, 1 se nahi.

EXAMPLE — Zero Se Ginti Shuru Hona — Confusing Par Zaroori

Agar `outcome_cols[0]` likhein, Python "`ndbi_before`" dega (list ka pehla item), na ki dusra. `outcome_cols[1]` dega "`ndbi_after`". Ye 0-se-shuru counting shuru mein ajeeb lagti hai (hum Hindi/English mein 1 se ginte hain), lekin practically har programming language mein yahi convention hai — jitni jaldi ye 'normal' lage, utna behtar. Ek acha tareeka yaad rakhne ka: index ka matlab hai 'pehle item se kitne steps door', aur pehla item khud se 0 steps door hai.

Lists mutable hote hain — matlab unme naye items add kiye ja sakte hain (`list.append(item)`), ya remove kiye ja sakte hain. BORDER OPTICS mein `merge_geocoded.py` file mein ek khaali list `rows = []` se shuru hoti hai, aur fir ek loop ke andar `rows.append(...)` se usme data dheere-dheere bhara jaata hai — hum ye poora pattern is PDF ke aakhri section mein detail se dekhenge.

6. Dictionaries — Key Aur Value Ke Jode

Ek dictionary (short mein 'dict') list se alag hai — list mein items sirf position (index) se access hote hain, lekin dictionary mein har value ka apna, custom naam (key) hota hai. Dictionary `{ }` (curly braces) mein likhi jaati hai, format `key: value`.

```
FILES = { "Arunachal Pradesh": "data/processed/arunachal_pradesh_geocoded.csv",
          "Sikkim": "data/processed/sikkim_geocoded.csv", "Uttarakhand":
          "data/processed/uttarakhand_geocoded.csv", "Himachal Pradesh":
          "data/processed/himachal_pradesh_geocoded.csv", }
```

Ye asli dictionary hai `merge_geocoded.py` se. Yahan har key ek state ka naam hai (jaise "Sikkim"), aur har value uski geocoded file ka path hai. List ki tarah index number se access karne ki jagah, dictionary mein hum key se access karte hain: `FILES["Sikkim"]` seedha "`data/processed/sikkim_geocoded.csv`" de dega — bina ye yaad rakhe ki Sikkim list mein kis position par hai.

PDF 1 mein humne ek dictionary already dekhi thi — `WINDOWS`, jisme keys the "`full_year`" aur "`summer`", aur values the ek aur dictionary (dictionary ke andar dictionary bhi ho sakti hai — `WINDOWS["full_year"]["before"]` deta ("`2021-01-01`", "`2022-01-01`")). Ye nested dictionary ka example hai, jo complex, organized data ko ek clean structure mein rakhne ke kaam aata hai.

EXAMPLE — Fridge Ke Labeled Shelves Ki Analogy

Socho ek fridge hai jisme har shelf par ek label chipka hai — 'Doodh', 'Sabzi', 'Fal'. Agar tujhe doodh chahiye, tu seedhe 'Doodh' label wali shelf khol degi, ye gin ke nahi ki 'doodh' teesri shelf par hai'. Dictionary bhi yahi karti hai — key (jaise 'Doodh') se seedha value (jaise doodh ki bottle) tak pahunch jaati hai, koi position count karne ki zaroorat nahi.

6.1 .items(), .keys(), .values() — Dictionary Ke Andar Loop Chalana

Jab humein poori dictionary ke har key-value pair par kuch karna ho, hum .items() use karte hain — jaisa merge_geocoded.py mein hota hai:

```
for state, path in FILES.items(): df = pd.read_csv(path)
```

Ye line dictionary ke har pair ko ek-ek karke nikaalti hai — pehli baar state = "Arunachal Pradesh", path = "data/processed/arunachal_pradesh_geocoded.csv", fir agli baar Sikkim ke liye, aur aise hi aage. Isko detail mein hum Section 8 (Loops) mein dekhenge.

7. Sets Aur Tuples — Do Aur Chhote Lekin Zaroori Types

7.1 Sets — Bina Duplicate, Bina Order Ke List

Ek set bhi curly braces { } mein likha jaata hai (dictionary jaisa dikhta hai, lekin colon : nahi hota, sirf values), aur iski do khaas properties hain: koi duplicate nahi (agar ek value do baar daalo bhi, set usko sirf ek baar rakhega), aur koi fixed order nahi.

```
MATCHED_STATUSES = { "matched", "matched (fallback query, no block)", "matched via  
Bhuvan (district-verified)", }
```

Ye asli set hai merge_geocoded.py se — teen alag tareeke jinse ek gaon 'successfully geocoded' maana jaata hai. Set ka use yahan .isin() check ke saath hota hai (agla section), jo dekhta hai ki kya kisi row ki value in teeno mein se koi ek hai.

7.2 Tuples — Ek Fixed, Na-Badalne-Wali List

Ek tuple round brackets () mein likha jaata hai, aur list jaisa hi dikhta hai — farak sirf itna hai ki tuple immutable hai, matlab ek baar ban jaaye toh usme koi change (add/remove/modify) nahi ho sakta.

```
"before": ("2021-01-01", "2022-01-01"), "after": ("2025-01-01", "2026-01-01"),
```

Ye wahi WINDOWS dictionary ka hissa hai jo PDF 1 mein dekha tha — har date-pair ek tuple hai. Tuple yahan isliye use hui hai kyunki 'before' aur 'after' dates fixed hain, research ke dauraan kabhi accidentally badalni nahi chahiye — tuple use karke ye

guarantee milti hai ki koi bhi baad ki line galti se inhe modify nahi kar sakti.

8. Operators — Compare Aur Combine Karna

Python mein values compare karne ke liye operators hote hain: `==` (barabar hai kya?, note — ek `=` assignment ke liye, do `==` comparison ke liye, ye ek common beginner confusion hai), `!=` (barabar nahi hai kya?), `>`, `<`, `>=`, `<=`. Logical combination ke liye: `and`, `or`, `not`.

```
matched["is_core_sample"] = state != "Himachal Pradesh"
```

Ye asli line hai `merge_geocoded.py` se. Yahan `state != "Himachal Pradesh"` ek comparison hai jo `True` ya `False` deta hai (agar `state` Himachal Pradesh nahi hai, toh `True`). Ye seedha ek naye column `is_core_sample` mein store ho jaata hai — isi tarah PDF 1 mein humne dekha tha ki Himachal Pradesh ke 7 gaon 'illustrative sample' hain, `core 251` ka hissa nahi.

9. If / Elif / Else — Faisla Lena

Conditionals Python ko 'faisla lene' dete hain — 'agar ye sach hai, toh ye karo, warna wo karo'. Keyword hai `if`, optional `elif` (matlab 'else if', aur conditions check karne ke liye), aur optional `else` (agar upar koi bhi condition sach na ho toh).

```
if count == 0: return None, 0  
composite = collection.median()
```

Ye wahi snippet hai jo humne indentation section mein dekha tha, ab logic ki taraf se dekhte hain: agar `count` (kitni images mili) 0 hai, function turant `None, 0` return kar deta hai (matlab 'koi data nahi mila'), aur neech ki lines (jo `composite` banati hain) bilkul nahi chalengi. Agar `count` 0 nahi hai, `if` block skip ho jaata hai aur seedha `composite` banane ka kaam shuru ho jaata hai.

Ternary (ek-line) conditional bhi Python mein possible hai, jaise `compute_border_distance.py` mein: `return right if left == "India" else left` — iska matlab hai 'agar `left` 'India' hai, toh `right` return karo, warna `left` return karo'. Ye normal multi-line if-else ka hi ek compact version hai, jab logic simple ho.

10. Loops — Ek Kaam Ko Baar-Baar Karna

Loop wo cheez hai jo Python ko itna powerful banati hai — 258 gaon ke liye ek hi calculation haath se 258 baar likhne ki jagah, ek loop likhte hain jo automatically har gaon ke liye repeat ho jaata hai. Sabse common loop hai `for`.

```
for i, row in villages.iterrows(): if all(pd.notna(row.get(c)) for c in outcome_cols): continue
point = ee.Geometry.Point([row["longitude"], row["latitude"]]) buffered_geom =
point.buffer(BUFFER_RADIUS_M)
```

Ye asli code hai `extract_satellite_data.py` se — poori extraction ka dil yahi loop hai. `villages.iterrows()` dataframe (agle section mein) ke har row ko ek-ek karke deta hai; `i` row ka index number hai, row us poore row ka data. Har iteration mein (har gaon ke liye), pehle check hota hai ki kya ye gaon already poora complete hai (agar haan, `continue` se seedha agle gaon par chale jaate hain, is gaon ko skip karke); agar nahi, uski latitude-longitude se ek point banta hai, fir uske around buffer.

`continue` ek special keyword hai jo loop ke current iteration ko turant rokke, seedha agle iteration par chala jaata hai — bahut useful jab kisi particular case ko skip karna ho bina poora loop rokke hue.

10.1 List/Dict Comprehension — Loop Ka Compact Version

Python mein ek loop ko ek hi line mein bhi likha ja sakta hai, agar wo simple ho — isko comprehension kehte hain. `holm_correction.py` mein ek example hai:

```
raw_p = [tests[n] for n in names]
```

Ye ek normal loop ka hi compact version hai — 'har naam `n` ke liye `names` list mein se, uski corresponding value `tests[n]` nikaalo, aur sab ko ek nayi list `raw_p` mein daal do'. Agar hum ise normal, multi-line loop mein likhte, toh 3-4 lines lagti; comprehension mein ek hi line mein ho jaata hai. Shuru mein padhne mein thoda tricky lag sakta hai, lekin pattern samajh aane ke baad ye bahut common Python style ban jaati hai.

10.2 While Loop — Jab Tak Ek Condition Sach Rahe

`for` loop ek fixed list/range par chalta hai. `while` loop tab chalta hai jab tak ek condition True rahe — BORDER OPTICS mein zyada common nahi hai (zyadatar `for` loops hi use hue hain, kyunki hume pata hota hai kitni baar repeat karna hai — 258 gaon, 14 districts, etc.), lekin concept yahan note kar lena zaroori hai kyunki aage kabhi mil sakta hai.

11. Functions — Reusable Instructions Ka Ek Packet

Ek function code ka ek chhota, named packet hai jo ek specific kaam karta hai, aur jise hum jitni baar chahein 'bula' (call kar) sakte hain, bina us kaam ke steps baar-baar dobara likhe. Function `def` keyword se define hota hai.

```
def other_country(row): left, right = row.get("ADM0_LEFT"), row.get("ADM0_RIGHT")
return right if left == "India" else left
```

Ye asli function hai `compute_border_distance.py` se. `def other_country(row):` batata hai ki hum ek naya function bana rahe hain, naam `other_country`, jo ek parameter (input) leta hai jise andar row ke naam se access kiya ja sakta hai. Andar, do variables ek saath assign ho rahe hain (`left, right = ...` — Python mein ek hi line mein multiple variables assign karne ka ek shortcut), aur function ke aakhir mein `return` se ek value wapas bheji jaati hai — jo bhi is function ko bulayega, use wahi value milegi.

EXAMPLE — Chai Banane Ki Machine Ki Analogy

Function ek chai-banane-ki-machine jaisi hai. Tu usme doodh, chai patti, cheeni daalti hai (ye parameters/inputs hain), machine andar-andar apna kaam karti hai (ye function ka body/logic hai), aur aakhir mein tujhe ek cup chai milti hai (ye return value hai). Tujhe har baar chai banane ka poora process dobara samajhne ki zaroorat nahi — bas machine ko sahi ingredients do, aur wo consistently same result de degi. BORDER OPTICS mein `ndbi_for_period()`, `lights_for_period()` jaise functions bilkul yahi karte hain — inko bas ek gaon ka location aur date-range do, wo NDBI ya lights nikaal ke de denge, chahe 1 baar bulao ya 258 baar.

Function ke andar function bulana bhi common hai — `extract_window()` apne andar `ndbi_for_period()` aur `lights_for_period()` dono ko bulata hai, har gaon ke liye. Ye modularity kehلاتa hai — bade kaam ko chhote, reusable pieces mein todna, jo har ek apna ek specific, testable kaam karta hai.

11.1 Multiple Return Values — Ek Saath Do Cheezein Wapas Bhejna

Python mein ek function ek se zyada value bhi wapas bhej sakta hai, comma se separate karke — jaise `holm_correction.py` mein:

```
def wilcoxon_ndbi_lights(path): ... return p_ndbi, p_lights # Function ko bulate waqt:
p_ndbi, p_lights = wilcoxon_ndbi_lights(path)
```

Function `p_ndbi` aur `p_lights` dono ek saath return karta hai (ye technically ek tuple return kar raha hai, jo humne Section 7.2 mein dekha), aur jab hum ise bulate hain, hum dono values ek saath, alag-alag variables mein receive kar sakte hain. Ye pattern BORDER OPTICS mein bahut baar dikhega, khaaskar jab NDBI aur lights dono ek saath process ho rahe hon.

12. Import — Doosron Ka Likha Hua Code Use Karna

Har baar poora code khud se likhne ki zaroorat nahi — Python mein hazaron libraries hain, jo doosre logon/organizations ne already likh ke publicly available kar di hain. Inko use karne ke liye, file ke sabse upar hum unko import karte hain.

```
import pandas as pd
import ee from scipy
import stats from dotenv
import load_dotenv
```

import pandas as pd ka matlab: pandas library import karo, lekin file mein use kar■■■ waqt pandas.read_csv(...) ki jagah short naam pd.read_csv(...) likhne do (as pd ek 'nickname' set kar raha hai — पूरी community isi nickname ko convention ki tarah follow karti hai). import ee Google Earth Engine ki library import karta hai (PDF 1 mein discuss kiya tha). from scipy import stats ka matlab: पूरी scipy library nahi, sirf uska ek chhota hissa (stats) import karo — jab पूरी library nahi chahiye, sirf ek specific part chahiye. from dotenv import load_dotenv isi pattern se ek specific function import kar raha hai, पूरा module nahi.

BORDER OPTICS ke har file mein jo libraries use hoti hain, wo hain: pandas (tables/dataframes ke liye, agla section), ee (Google Earth Engine), scipy aur statsmodels (statistics ke liye, PDF 3), geopandas aur shapely (geographic/map data ke liye), aur argparse (command-line se options lene ke liye, jo hum agle section mein dekhenge).

13. Pandas DataFrame — BORDER OPTICS Ka Sabse Zaroori Tool

Ab sabse important concept par aate hain, jo BORDER OPTICS ke har single analysis/acquisition file mein hai: pandas library ka DataFrame. Simple shabdon mein, DataFrame ek table hai — bilkul Excel spreadsheet jaisi — jisme rows (har row ek gaon, jaise) aur columns (jaise 'village', 'ndbi_before', 'district') hote hain, aur Python mein hum is पूरी table ko ek hi variable mein rakh sakte hain aur uske saath पूरी tarah operate kar sakte hain.

```
df = pd.read_csv("data/processed/border_optics_master_villages.csv")
```

Ye ek line पूरी CSV file (comma-separated table) ko padh ke ek DataFrame mein badal deti hai, variable df mein store karke. Ab df ke paas 258 rows (har gaon ek row) aur columns jaise village, district, latitude, longitude hain.

13.1 Column Select Karna

```
villages["latitude"] # Ek column, sab rows villages[["village_id", "village", "state"]] #  
Multiple columns, ek naye chhote DataFrame mein
```

Single square brackets ["col"] ek column deta hai. Double square brackets [["col1", "col2"]] (ek list of column names, list ke andar) multiple columns deta hai — ye exact wahi pattern hai jo merge_geocoded.py mein use hota hai final columns select karne ke liye.

13.2 Boolean Filtering — Rows Chunna Ek Condition Se

```
matched = df[df["geocode_status"].isin(MATCHED_STATUSES)].copy()
```

Ye ek bahut common aur zaroori pattern hai. `df["geocode_status"].isin(MATCHED_STATUSES)` har row ke liye ek True/False value banata hai — True agar us row ka `geocode_status` humari `MATCHED_STATUSES` set mein maujood hai. Fir `df[...]` (poore True/False list ke saath) sirf wahi rows rakhta hai jinka value True hai — baaki sab rows hata di jaati hain. `.copy()` ek explicit copy banata hai (iske bina kabhi-kabhi Python confusion warning deta hai ki tu original data modify kar rahi hai ya copy — `.copy()` lagana isko clean/explicit bana deta hai).

EXAMPLE — Chhalni (Sieve) Ki Analogy

Socho ek chhalni hai jisse tu chawal ke daane chhaan rahi hai, sirf achhe daane rakhne ke liye. Boolean filtering bilkul yahi karti hai — poori dataframe ko ek condition ke through 'chhaanti' hai, sirf wo rows rakhti hai jo condition True dete hain. Ye ek-baar mein, poori table par, bina loop likhe, ho jaata hai — jo Python ki bahut badi strength hai.

13.3 Naye Columns Banana

```
matched["state"] = state matched["is_core_sample"] = state != "Himachal Pradesh"
```

Agar tu ek naam pe value assign karti hai jo pehle se column nahi hai, Python automatically ek naya column bana deta hai, poori dataframe mein us value se bhar deta hai (ya, agar right side khud ek column/calculation hai, row-by-row correct value se bharta hai).

13.4 .loc[] Aur .at[] — Specific Jagah Value Set/Get Karna

```
villages.at[i, "ndbi_before"] = ndbi_before india_segments.loc[seg_idx, "other_country"]
```

`.at[row, "column"]` ek single, specific cell ki value set ya get karne ke liye use hota hai — jaise `extract_satellite_data.py` ke loop mein, har gaon ka result seedha uske row-index aur column-name se store hota hai. `.loc[]` isi jaisa hai lekin thoda zyada flexible (multiple rows/columns bhi le sakta hai).

13.5 .merge(), .concat(), .apply() — Tables Ko Jodna Aur Process Karna

```
result = pd.concat([villages, nearest_info], axis=1) merged =  
results.merge(distances[["village_id", "distance_to_border_km"]], on="village_id",  
how="inner") india_segments["other_country"] = india_segments.apply(other_country,  
axis=1)
```

`pd.concat([...], axis=1)` do dataframes ko side-by-side jodta hai (columns add hote hain — `axis=1` ka matlab hai 'columns ki taraf', agar `axis=0` hota toh rows neeche jud jaate). `.merge()` SQL-jaisa join hai — do tables ko ek shared column (`on="village_id"`) ke

through jodta hai, sirf wahi rows rakhta hai jinka match dono taraf ho (how="inner"). .apply() ek function (jaise humne Section 11 mein dekha other_country) ko har row par chalata hai, ek loop likhne ki zaroorat ke bina — axis=1 ka matlab hai 'har row par' (agar axis=0 hota, har column par chalta).

13.6 .dropna() — Missing Values Hatana

```
valid_ndbi = core.dropna(subset=["ndbi_before", "ndbi_after"])
```

PDF 1 mein humne dekha tha ki agar kisi gaon ke liye koi valid image nahi milti, uski value None/null store hoti hai, zero nahi. .dropna(subset=[...]) un rows ko hata deta hai jinke specified columns mein koi missing (null) value hai — taaki statistics test (PDF 3) sirf complete, valid data par chale, missing data ko galti se 0 maan ke nahi.

13.7 .to_csv() — Wapas File Mein Save Karna

```
master.to_csv("data/processed/border_optics_master_villages.csv", index=False)
```

Poori processed dataframe ko wapas ek CSV file mein save karta hai. index=False ka matlab hai: dataframe ka internal row-number (0, 1, 2...) file mein save mat karo, sirf asli columns save karo — warna ek extra, unwanted column file mein aa jaata.

14. F-Strings — Variables Ko Text Ke Andar Daalna

Jab humein ek message print karna ho jisme ek variable ki value bhi dikhani ho, hum f-strings use karte hain — ek string jiske shuru mein f lagta hai, aur andar { } mein koi bhi variable/expression daal sakte hain.

```
print(f"Merged {len(master)} villages into  
data/processed/border_optics_master_villages.csv")
```

Ye asli line hai merge_geocoded.py se. {len(master)} ke jagah run-time par asli number aa jaata hai (jaise 258) — len(master) batata hai dataframe mein kitni rows hain. Result print hota hai: "Merged 258 villages into data/processed/border_optics_master_villages.csv". F-strings ka fayda ye hai ki hume manually string ko todke, variable ko '+' se jodna nahi padta — sab ek hi, readable line mein ho jaata hai.

Zyada advanced formatting bhi possible hai, jaise holm_correction.py mein: f'{name:32s} {raw:14.6e}' — :32s ka matlab hai 'ise 32 characters ki width mein, text ki tarah, left-align karke print karo' (taaki columns neatly line up hon), aur :14.6e ka matlab hai 'ise 14 characters ki width mein, scientific notation mein, 6 decimal places ke saath print karo'. Ye alignment tricks BORDER OPTICS ke output tables ko readable banane ke kaam aati hain.

15. Try/Except — Jab Kuch Galat Ho Sakta Hai

Kabhi-kabhi ek line fail ho sakti hai (jaise internet connection na ho, ya file na mile). Agar aisa hota aur hum kuch nahi karte, poora program crash ho jaata. try/except Python ko batata hai: 'ye try karo, aur agar fail ho jaaye, crash mat ho, iski jagah ye doosra kaam karo'.

```
def init_ee(): project = os.environ.get("EE_PROJECT") try: ee.Initialize(project=project) except Exception: ee.Authenticate() ee.Initialize(project=project)
```

Ye asli function hai `extract_satellite_data.py` se. Pehle Google Earth Engine ko seedha Initialize karne ki koshish hoti hai (agar pehle se authenticated ho). Agar ye fail ho jaaye (except Exception: — koi bhi tarah ki error), toh Python crash nahi hota, balki except block chal jaata hai — pehle `Authenticate()` (login process) chalta hai, fir dobara `Initialize()` try hota hai. Ye pattern research code mein bahut common hai jahan external services (internet, APIs) involve hoti hain, jo hamesha 100% predictable nahi hoti.

16. if __name__ == "__main__": — Ek Common Pattern Jo Har File Mein Dikhega

BORDER OPTICS ki bahut saari files ke bilkul aakhir mein ye line milegi:

```
if __name__ == "__main__": main()
```

Iska matlab samajhna zaroori hai kyunki ye PDF 6 se har file mein baar-baar dikhega. Har Python file ko Python ek special, automatic variable `__name__` deta hai. Agar file ko seedha run kiya jaaye (jaise `python holm_correction.py`), `__name__` ki value automatically `"__main__"` ho jaati hai. Lekin agar isi file ko kisi doosri file se import kiya jaaye (jaise `from holm_correction import wilcoxon_ndbi_lights`), toh `__name__` file ka naam hota hai, `"__main__"` nahi.

EXAMPLE — Kyun Zaroori Hai Ye Pattern?

Ye pattern ek file ko do tareeke se use karne deta hai: (1) seedha, standalone script ki tarah run karna (jab poora analysis khud chalana ho), ya (2) uske functions ko doosri files mein reuse karna (bina us file ka poora, standalone 'main' logic automatically chalaye). Jaise, koi doosri file `holm_correction.py` ka sirf `wilcoxon_ndbi_lights()` function use karna chahe, bina poori Holm-correction analysis dobara chalaye — ye `if __name__ == "__main__":` guard exactly isko possible banata hai.

17. Sab Kuch Jod Ke — Ek Poori Asli File Line-By-Line

Ab jab humne har concept alag-alag dekh liya, chalo ek poori, chhoti, asli BORDER OPTICS file (merge_geocoded.py, sirf 37 lines) shuru se aakhir tak, line-by-line padhte hain — dono nazariye se: ye line research mein kya kar rahi hai, aur iski Python syntax kya hai.

```
import pandas as pd
```

Purpose: pandas library import kar rahe hain, table/dataframe operations ke liye.
Syntax: import X as Y — poora module import karo, chhote naam pd se refer karo.

```
FILES = { "Arunachal Pradesh": "data/processed/arunachal_pradesh_geocoded.csv", ... }
```

Purpose: Har state ki geocoded CSV file ka path store kar rahe hain, ek jagah, taaki aage loop mein use ho sake. **Syntax:** dictionary literal — {key: value, ...}, keys state names (strings), values file paths (strings).

```
MATCHED_STATUSES = {"matched", ...}
```

Purpose: Kaun se geocode-status values 'successfully matched' maane jaayenge, ye define kar rahe hain. **Syntax:** set literal — {value1, value2, ...}, koi colon nahi (dictionary se yahi farak hai).

```
rows = []
```

Purpose: Ek khaali list bana rahe hain jisme har state ka processed data dheere-dheere daala jaaega. **Syntax:** khaali list literal — [].

```
for state, path in FILES.items():
```

Purpose: Har state (aur uski file-path) par, ek-ek karke, loop chala rahe hain. **Syntax:** for a, b in dict.items(): — dictionary ke har key-value pair ko do variables mein unpack karte hue loop.

```
df = pd.read_csv(path)
```

Purpose: Us state ki CSV file ko padh ke ek dataframe bana rahe hain. **Syntax:** function call, result ek naye variable df mein store; indented (4 spaces), matlab ye loop ke andar hai, har state ke liye alag se chalega.

```
matched = df[df["geocode_status"].isin(MATCHED_STATUSES)].copy()
```

Purpose: Sirf wahi rows rakh rahe hain jinka geocode-status humari 'matched' list mein hai — baaki (failed geocoding) discard. **Syntax:** boolean filtering (Section 13.2), chained saath .copy().

```
matched["state"] = state
```

Purpose: Ek naya column bana rahe hain jisme har row ka state-naam likha ho (kyunki original file mein shayad state-naam nahi tha, ye humein pata hai loop ke current state se). **Syntax:** naya column assignment (Section 13.3).

```
matched["is_core_sample"] = state != "Himachal Pradesh"
```

Purpose: Flag kar rahe hain ki ye state core statistical sample ka hissa hai ya sirf illustrative (Himachal Pradesh). **Syntax:** comparison operator (!=) ka result seedha column mein store — Boolean value (True/False).

```
rows.append(matched[...])
```

Purpose: Is state ka processed data, khaas columns select karke, पूरी list mein add kar rahe hain, taaki baad mein sab states ka data ek saath jod sakein. **Syntax:** list.append(item) — Section 5 mein dekha gaya list method.

```
master = pd.concat(rows, ignore_index=True)
```

Purpose: Sab states ka data (jo abhi tak rows list mein alag-alag tha) ek hi, badi dataframe mein jod rahe hain. **Syntax:** pd.concat(list_of_dataframes); ignore_index=True ka matlab hai naye, fresh row-numbers 0,1,2... use karo, purane alag-alag files ke row-numbers nahi.

```
master["village_id"] = range(1, len(master) + 1)
```

Purpose: Har gaon ko ek unique, sequential ID de rahe hain (1, 2, 3, ...). **Syntax:** range(start, stop) ek sequence of numbers generate karta hai, start se stop-1 tak (isliye + 1 lagaya hai, taaki last village bhi included ho).

```
master.to_csv("data/processed/border_optics_master_villages.csv", index=False)
```

Purpose: Final, merged, 258-village master table ko permanently CSV file mein save kar rahe hain, taaki aage ki har file (geocoding se lekar analysis tak) yahi ek file use kare. **Syntax:** Section 13.7 mein dekha gaya .to_csv() method.

```
print(f"Merged {len(master)} villages into  
data/processed/border_optics_master_villages.csv")
```

Purpose: Confirmation message print kar rahe hain, taaki jab ye script chale, humein turant pata chale kitne villages successfully merge hue. **Syntax:** f-string (Section 14) ke saath print.

Bas — 37 lines mein, ye पूरी file 4 alag states ki geocoded files ko padhti hai, sirf successfully-matched villages rakhti hai, unko ek consistent structure deti hai, aur ek single, clean master table save karti hai — jo aage har analysis ki neev banti hai. Agar tujhe ye पूरी file ab clearly samajh aa rahi hai, matlab is PDF ka kaam ho gaya — tu

ab BORDER OPTICS ke asli code files padhne ke liye ready hai.

Is PDF Ka Recap — Ab Tak Kya Seekha

→ Python top-se-neeche, line-by-line chalta hai — comments (`#`, `"""..."""`) insaan ke liye hote hain, Python unhe ignore karta hai.

→ Variables (jaise `BUFFER_RADIUS_M = 500`) data ko naam dete hain — `int`, `float`, `str`, `bool`, `None` basic types hain.

→ Indentation (4 spaces) Python mein blocks define karti hai — koi curly braces nahi, sirf whitespace.

→ Lists `[ ]` ordered hain, index 0 se shuru; dictionaries `{ key: value }` naam se access hoti hain; sets `{ }` duplicates nahi rakhte; tuples `( )` immutable hain.

→ `if/elif/else` se faisla; `for` loops se repetition; comprehensions se compact loops.

→ Functions (`def`) reusable code packets hain, multiple values bhi return kar sakte hain.

→ `import` se libraries use hoti hain — `pandas` sabse zaroori hai, `dataframe` (table) ke liye.

→ `DataFrame` mein filtering (`df[condition]`), naye columns, `.loc/.at`, `.merge/.concat/.apply`, aur `.dropna()` sabse common operations hain.

→ f-strings (`f"...{variable}..."`) text ke andar values daalte hain.

→ `try/except` errors ko gracefully handle karta hai; `if __name__ == "__main__":` file ko `run-vs-import` dono tareeke se use karne deta hai.

Aage kya: PDF 3 mein hum Statistics ki duniya mein jaayenge — `p-value`, `mean/median/correlation`, `confidence interval`, `hypothesis testing` — bilkul zero se, examples ke saath. PDF 4 mein Economics/Causal Inference — `control group`, `Difference-in-Differences`, `counterfactual`, `parallel trends`. Fir PDF 5 mein poori VVP-I policy story aur BORDER OPTICS ka overall research design (Big Picture). PDF 6 se asli code files khulna shuru honge, geocoding se — aur tab, is PDF mein seekhi har cheez turant kaam aayegi.

Jab ready ho agle PDF ke liye, bas bol dena.